Phase 2 Notes: Array Queues Using a Single Pointer and Two Pointers

1. Current Progress Before Phase 2

Before Phase 2, you already learned the basic idea of a queue.

A queue follows this rule:

First In, First Out

This is called **FIFO**.

That means:

> The first value inserted into the queue is the first value removed from the queue.

You also learned these basic queue ideas:

Idea	Simple Meaning
`enqueue`	Insert a value into the queue
`dequeue`	Remove a value from the queue
`front`	The side where deletion happens
`rear`	The side where insertion happens
FIFO	First inserted value leaves first

Phase 2 now asks an important question:

> How can we store a queue inside an array?

---

2. Main Goal of Phase 2

In Phase 2, we learn two ways to build a queue using an array:

1. **Single-pointer queue**
2. **Two-pointer queue**

The main lesson is:

A single-pointer queue is simple, but deletion is slow. A two-pointer queue is better because deletion becomes fast.

---

3. What Is an Array Queue?

An **array queue** stores queue values inside an array.

Example:

```
int q[5];
```

This creates an array with 5 spaces.

```
Index:  0   1   2   3   4
Array: [ ] [ ] [ ] [ ] [ ]
```

Each position has an index.

The first index is `0`.

The last index is:

```
size - 1
```

So if the size is `5`, the last index is:

```
5 - 1 = 4
```

---

4. Why Do We Need Pointers or Index Variables?

In an array queue, we need to know two things:

1. Where to insert the next value
2. Where to delete the next value

To track these positions, we use index variables.

These variables are often called pointers in queue explanations, but in this array version they are **not real memory address pointers**.

They are simply **array indexes**.

Example:

```
front = 0
rear = 2
```

This means `front` and `rear` are positions inside the array.

---

5. Two Designs in Phase 2

Phase 2 compares these two designs:

Design	Variables Used	Main Idea
Single-pointer queue	Only `rear`	Insert is easy, but delete is slow
Two-pointer queue	`front` and `rear`	Insert and delete are both fast

---

Part A: Single-Pointer Array Queue

6. What Is a Single-Pointer Queue?

A **single-pointer queue** uses only one index variable:

```
rear
```

The `rear` variable keeps track of the last inserted element.

At the start:

```
rear = -1;
```

This means the queue is empty because no value has been inserted yet.

---

7. Why Does `rear` Start at `-1`?

Array indexes start from `0`.

So before the first value is inserted, `rear` is set to `-1`.

That means:

```
No valid value exists yet.
```

When the first value is inserted, `rear` moves from `-1` to `0`.

---

8. Single-Pointer Insertion

Insertion in the single-pointer queue is easy.

To insert a value:

1. Move `rear` one step forward.
2. Store the new value at `queue[rear]`.

Code:

```
void insert(int queue[], int& rear, int x) {
    rear++;
    queue[rear] = x;
}
```

---

9. Single-Pointer Insertion Dry Run

Start:

```
rear = -1
Array: [ ] [ ] [ ] [ ] [ ]
```

Insert 5

```
insert(queue, rear, 5);
```

`rear` moves from `-1` to `0`.

```
Index:  0   1   2   3   4
Array: [5] [ ] [ ] [ ] [ ]
rear = 0
```

Insert 10

```
insert(queue, rear, 10);
```

`rear` moves from `0` to `1`.

```
Index:  0    1   2   3   4
Array: [5] [10] [ ] [ ] [ ]
rear = 1
```

Insert 15

```
insert(queue, rear, 15);
```

`rear` moves from `1` to `2`.

```
Index:  0    1    2   3   4
Array: [5] [10] [15] [ ] [ ]
rear = 2
```

The queue now contains:

```
5, 10, 15
```

---

10. Time Complexity of Single-Pointer Insertion

Single-pointer insertion is fast.

It only does two small actions:

1. Move `rear` forward.
2. Store the value.

So the time complexity is:

```
O(1)
```

`O(1)` means constant time.

The operation does not become slower when the queue has more values.

---

11. Single-Pointer Deletion

A queue must delete from the **front**.

In the single-pointer design, the front is always kept at index `0`.

So deletion always removes:

```
queue[0]
```

Example:

```
Index:  0    1    2
Array: [5] [10] [15]
rear = 2
```

The first value is `5`, so `5` must be deleted first.

That follows FIFO.

---

12. The Problem After Deleting from Index 0

If we delete `5`, the array becomes like this:

```
Index:  0    1    2
Array: [ ] [10] [15]
```

Now there is an empty space at the front.

But in the single-pointer design, the front must stay at index `0`.

So we must shift all remaining elements one step to the left.

```
10 moves to index 0
15 moves to index 1
```

After shifting:

```
Index:   0    1   2
Array: [10] [15] [ ]
rear = 1
```

Now the queue is correct again.

---

13. Single-Pointer Deletion Code

```
int removeItem(int queue[], int& rear) {
    int deletedItem = queue[0];

    for (int i = 0; i < rear; i++) {
        queue[i] = queue[i + 1];
    }

    rear--;
    return deletedItem;
}
```

---

14. Explanation of Single-Pointer Deletion Code

Step 1: Store the deleted item

```
int deletedItem = queue[0];
```

This saves the value at the front.

Step 2: Shift elements left

```
for (int i = 0; i < rear; i++) {
    queue[i] = queue[i + 1];
}
```

This loop moves every remaining value one position left.

Step 3: Decrease `rear`

```
rear--;
```

The queue has one fewer value now, so `rear` must move backward.

Step 4: Return the deleted value

```
return deletedItem;
```

This gives back the value that was removed.

---

15. Single-Pointer Deletion Dry Run

Start:

```
Index:  0    1    2
Array: [5] [10] [15]
rear = 2
```

Call:

```
removeItem(queue, rear);
```

Step 1: Delete index 0

```
Deleted value = 5
```

Step 2: Shift values left

```
10 moves from index 1 to index 0
15 moves from index 2 to index 1
```

Step 3: Decrease `rear`

```
rear moves from 2 to 1
```

Final array:

```
Index:   0    1   2
Array: [10] [15] [ ]
rear = 1
```

The queue now contains:

```
10, 15
```

---

16. Why Single-Pointer Deletion Is Slow

The problem is this loop:

```
for (int i = 0; i < rear; i++) {
    queue[i] = queue[i + 1];
}
```

If the queue has many elements, many elements must move.

Example:

If the queue has 1000 elements, deleting one value may require shifting 999 elements.

So deletion is:

```
O(n)
```

`O(n)` means the time depends on the number of elements.

More elements means more work.

---

17. Single-Pointer Queue Summary

Operation	What Happens	Time Complexity
Insert	Move `rear` and store value	`O(1)`
Delete	Remove index `0` and shift elements left	`O(n)`

Main problem:

> Single-pointer deletion is slow because all remaining elements must shift left.

Part B: Two-Pointer Array Queue

18. Why Do We Need a Two-Pointer Queue?

The single-pointer queue has a problem:

> Deletion requires shifting.

The two-pointer queue solves this problem.

Instead of shifting elements, we move another index variable called `front`.

So the two-pointer queue uses:

```
front
rear
```

---

19. Meaning of `front` and `rear`

Pointer	Simple Meaning
`rear`	Points to the last inserted element
`front`	Points before the first valid element

Important:

In this design, `front` does **not** directly point to the first valid element.

Instead:

```
first valid element = front + 1
```

---

20. Starting State of a Two-Pointer Queue

At the beginning:

```
front = -1;
rear = -1;
```

This means the queue is empty.

Because:

```
front == rear
```

So the empty condition is:

```
front == rear
```

---

21. Two-Pointer Queue Structure

A two-pointer array queue can be represented using a structure:

```
struct Queue {
    int size;
    int front;
    int rear;
    int *Q;
};
```

---

22. Meaning of Each Structure Member

Member	Simple Meaning
`size`	Maximum number of values the queue can store
`front`	Tracks the deletion side
`rear`	Tracks the insertion side
`Q`	Points to the array that stores values

Again, remember:

> In this array queue, `front` and `rear` are index values, not memory addresses.

---

23. Empty Condition in a Two-Pointer Queue

The queue is empty when:

```
front == rear
```

Example 1:

```
front = -1
rear = -1
```

The queue is empty.

Example 2:

```
front = 2
rear = 2
```

The queue is also empty because there are no valid elements between `front` and `rear`.

---

24. Full Condition in a Linear Two-Pointer Array Queue

The queue is full when `rear` reaches the last valid index.

The last valid index is:

```
size - 1
```

So the full condition is:

```
rear == size - 1
```

Example:

```
size = 5
last index = 4
```

So the queue is full when:

```
rear = 4
```

---

25. Two-Pointer Enqueue Logic

To insert a value:

1. Check whether the queue is full.
2. If it is full, show a full message.
3. If it is not full, move `rear` one step forward.
4. Store the value at `Q[rear]`.

Code:

```
void enqueue(Queue* q, int x) {
    if (q->rear == q->size - 1) {
        cout << "Queue is Full";
    } else {
        q->rear++;
        q->Q[q->rear] = x;
    }
}
```

---

26. Two-Pointer Enqueue Dry Run

Start:

```
front = -1
rear  = -1
Array: [ ] [ ] [ ] [ ] [ ]
```

Enqueue 10

```
enqueue(&q, 10);
```

`rear` moves from `-1` to `0`.

```
Index:   0   1   2   3   4
Array: [10] [ ] [ ] [ ] [ ]
front = -1
rear  = 0
```

Enqueue 20

```
enqueue(&q, 20);
```

`rear` moves from `0` to `1`.

```
Index:   0    1   2   3   4
Array: [10] [20] [ ] [ ] [ ]
front = -1
rear  = 1
```

The queue now contains:

```
10, 20
```

---

27. Time Complexity of Two-Pointer Enqueue

Two-pointer enqueue only does two main actions:

1. Move `rear` forward.
2. Store the value.

So enqueue is:

```
O(1)
```

---

28. Two-Pointer Dequeue Logic

To delete a value:

1. Check whether the queue is empty.
2. If it is empty, show an empty message.
3. If it is not empty, move `front` one step forward.
4. Return the value at `Q[front]`.

Code:

```
int dequeue(Queue* q) {
    int x = -1;

    if (q->front == q->rear) {
        cout << "Queue is Empty";
    } else {
        q->front++;
        x = q->Q[q->front];
    }

    return x;
}
```

---

29. Two-Pointer Dequeue Dry Run

Suppose the queue contains:

```
10, 20
```

Array:

```
Index:   0    1
Array: [10] [20]
```

Pointers:

```
front = -1
rear  = 1
```

Call:

```
dequeue(&q);
```

Step 1: Check empty

```
front == rear?
-1 == 1? No
```

The queue is not empty.

Step 2: Move `front`

```
front moves from -1 to 0
```

Step 3: Return value

```
Q[front] = Q[0] = 10
```

So `10` is removed.

Now:

```
front = 0
rear  = 1
```

The queue logically contains:

```
20
```

---

30. Why Two-Pointer Deletion Is Fast

In the two-pointer queue, deletion does not shift elements.

It only moves `front` forward.

That means deletion is:

```
O(1)
```

This is faster than the single-pointer queue, where deletion is `O(n)`.

---

31. Logical Deletion

In a two-pointer queue, when a value is deleted, the value may still physically remain inside the array.

Example before deletion:

```
front = -1
rear = 2
Array = [10, 20, 30]
```

After one dequeue:

```
front = 0
rear = 2
Array may still be [10, 20, 30]
```

But the logical queue is now:

```
20, 30
```

Why?

Because valid elements are from:

```
front + 1
```

to:

```
rear
```

So the valid range is now:

```
1 to 2
```

That gives:

```
Q[1] = 20
Q[2] = 30
```

The old value `10` is ignored because `front` has moved past it.

This is called **logical deletion**.

---

32. Valid Range in a Two-Pointer Queue

In this design:

```
front points before the first valid element
rear points at the last valid element
```

So the valid queue elements are found from:

```
front + 1
```

to:

```
rear
```

Example:

```
front = 0
rear = 3
Array = [10, 20, 30, 40]
```

Valid range:

```
front + 1 = 1
rear = 3
```

So the valid queue is:

```
20, 30, 40
```

The value at index `0` is no longer part of the queue.

---

33. Full Dry Run of Two-Pointer Queue

Start:

```
front = -1
rear = -1
Queue is empty
```

Step 1: Enqueue 10

```
rear moves from -1 to 0
Q[0] = 10
```

Queue:

```
10
```

Pointers:

```
front = -1
rear = 0
```

Step 2: Enqueue 20

```
rear moves from 0 to 1
Q[1] = 20
```

Queue:

```
10, 20
```

Pointers:

```
front = -1
rear = 1
```

Step 3: Dequeue

```
front moves from -1 to 0
Q[0] = 10 is returned
```

Queue:

```
20
```

Pointers:

```
front = 0
rear = 1
```

The value `10` leaves first because it entered first.

This proves FIFO.

---

34. Two-Pointer Queue with A, B, and C

Start:

```
front = -1
rear = -1
Queue is empty
```

Enqueue A

```
rear moves from -1 to 0
A is stored at index 0
```

Queue:

```
A
```

Pointers:

```
front = -1
rear = 0
```

Enqueue B

```
rear moves from 0 to 1
B is stored at index 1
```

Queue:

```
A, B
```

Pointers:

```
front = -1
rear = 1
```

Enqueue C

```
rear moves from 1 to 2
C is stored at index 2
```

Queue:

```
A, B, C
```

Pointers:

```
front = -1
rear = 2
```

Dequeue

```
front moves from -1 to 0
A is removed
```

Queue:

```
B, C
```

Pointers:

```
front = 0
rear = 2
```

A entered first, so A leaves first.

This proves FIFO again.

---

35. Dry Run Table for A, B, and C

Step	Operation	`front` Meaning	`rear` Meaning	Logical Queue
1	Start	Same as rear	Same as front	Empty
2	Enqueue A	Before first element	Points at A	A
3	Enqueue B	Before first element	Points at B	A, B
4	Enqueue C	Before first element	Points at C	A, B, C
5	Dequeue	Moves to A position	Points at C	B, C

Part C: Comparing Single-Pointer and Two-Pointer Queues

36. Main Difference

The main difference is how deletion works.

Design	Deletion Method
Single-pointer queue	Delete index `0`, then shift all elements left
Two-pointer queue	Move `front` forward

---

37. Complexity Comparison

Operation	Single-Pointer Queue	Two-Pointer Queue
Enqueue / Insert	`O(1)`	`O(1)`
Dequeue / Delete	`O(n)`	`O(1)`

The two-pointer queue is better because both insertion and deletion are fast.

---

38. Feature Comparison

Feature	Single-Pointer Queue	Two-Pointer Queue
Uses `rear`	Yes	Yes
Uses `front`	No	Yes
Insert at rear	Yes	Yes
Delete from front	Yes	Yes
Needs shifting after deletion	Yes	No
Deletion speed	Slow	Fast
Better for implementation	No	Yes

---

39. Important Weakness of the Two-Pointer Linear Queue

The two-pointer queue is faster than the single-pointer queue.

But it still has one weakness.

In this simple linear array version, `rear` only moves forward.

The full condition is:

```
rear == size - 1
```

That means when `rear` reaches the last index, the queue is considered full.

This can later cause memory wastage because deleted spaces at the beginning may not be reused.

This problem is not fully solved in Phase 2.

It is introduced later when studying linear queue drawbacks and circular queues.

For Phase 2, remember:

> The two-pointer design is faster than the single-pointer design, but it is still a linear array queue.

Part D: Code Examples

40. Single-Pointer Insert Function

```
void singleInsert(int q[], int& rear, int x) {
    rear++;
    q[rear] = x;
}
```

Simple meaning:

1. Move `rear` forward.
2. Store `x` at that position.

---

41. Single-Pointer Delete Function

```
int singleDelete(int q[], int& rear) {
    if (rear == -1) {
        return -1;
    }

    int deleted = q[0];

    for (int i = 0; i < rear; i++) {
        q[i] = q[i + 1];
```

```
    }

    rear--;
    return deleted;
}
```

Simple meaning:

1. If `rear == -1`, the queue is empty.
2. Save the value at index `0`.
3. Shift remaining elements left.
4. Move `rear` backward.
5. Return the deleted value.

---

42. Two-Pointer Queue Structure

```
struct TwoPointerQueue {
    int size;
    int front;
    int rear;
    int *Q;
};
```

Simple meaning:

Member	Meaning
`size`	Capacity of the queue
`front`	Position before the first valid element
`rear`	Position of the last inserted element
`Q`	Dynamic array that stores values

---

43. Two-Pointer Enqueue Function

```
void enqueue(TwoPointerQueue *q, int x) {
    if (q->rear == q->size - 1) {
        cout << "Two-pointer queue is full\n";
    } else {
        q->rear++;
        q->Q[q->rear] = x;
```

```
    }
}
```

Simple meaning:

1. If `rear` is at the last index, the queue is full.
2. Otherwise, move `rear` forward.
3. Store the new value.

---

44. Two-Pointer Dequeue Function

```
int dequeue(TwoPointerQueue *q) {
    if (q->front == q->rear) {
        return -1;
    }

    q->front++;
    return q->Q[q->front];
}
```

Simple meaning:

1. If `front == rear`, the queue is empty.
2. Otherwise, move `front` forward.
3. Return the value at the new `front` position.

---

45. Complete Beginner Program

```
#include <iostream>
using namespace std;

void singleInsert(int q[], int& rear, int x) {
    rear++;
    q[rear] = x;
}

int singleDelete(int q[], int& rear) {
    if (rear == -1) {
        return -1;
    }

    int deleted = q[0];
```

```
    for (int i = 0; i < rear; i++) {
        q[i] = q[i + 1];
    }

    rear--;
    return deleted;
}

struct TwoPointerQueue {
    int size;
    int front;
    int rear;
    int *Q;
};

void enqueue(TwoPointerQueue *q, int x) {
    if (q->rear == q->size - 1) {
        cout << "Two-pointer queue is full\n";
    } else {
        q->rear++;
        q->Q[q->rear] = x;
    }
}

int dequeue(TwoPointerQueue *q) {
    if (q->front == q->rear) {
        return -1;
    }

    q->front++;
    return q->Q[q->front];
}

int main() {
    int a[5], rear = -1;

    singleInsert(a, rear, 5);
    singleInsert(a, rear, 10);
    singleInsert(a, rear, 15);

    cout << "Single-pointer deleted: " << singleDelete(a, rear) << "\n";

    TwoPointerQueue q;
    q.size = 5;
    q.front = q.rear = -1;
    q.Q = new int[q.size];
```

```
    enqueue(&q, 10);
    enqueue(&q, 20);

    cout << "Two-pointer deleted: " << dequeue(&q) << "\n";

    delete[] q.Q;

    return 0;
}
```

---

46. Program Output

```
Single-pointer deleted: 5
Two-pointer deleted: 10
```

---

47. Program Explanation

First Part: Single-Pointer Queue

```
int a[5], rear = -1;
```

This creates a simple array queue and starts `rear` at `-1`.

Then:

```
singleInsert(a, rear, 5);
singleInsert(a, rear, 10);
singleInsert(a, rear, 15);
```

The array becomes:

```
[5, 10, 15]
```

Then:

```
singleDelete(a, rear)
```

This deletes `5`, shifts `10` and `15` left, and returns `5`.

Second Part: Two-Pointer Queue

```
TwoPointerQueue q;
q.size = 5;
q.front = q.rear = -1;
q.Q = new int[q.size];
```

This creates a two-pointer queue.

Then:

```
enqueue(&q, 10);
enqueue(&q, 20);
```

The queue becomes:

```
10, 20
```

Then:

```
dequeue(&q)
```

This removes and returns `10` by moving `front` forward.

No shifting is needed.

Part E: Common Beginner Mistakes

48. Mistake 1: Forgetting to Shift in the Single-Pointer Queue

Wrong idea:

> Delete index `0` and do nothing else.

Problem:

This leaves an empty gap at the front.

Correct idea:

> After deleting index `0`, shift all remaining elements one step left.

---

49. Mistake 2: Forgetting `rear--` After Shifting

After deletion, the queue has one fewer value.

So `rear` must move backward.

Correct:

```
rear--;
```

If you forget this, the queue may think it has more values than it really has.

---

50. Mistake 3: Thinking `front` Points Directly to the First Element

In this design, `front` points before the first valid element.

So the first valid element is:

```
front + 1
```

Example:

```
front = 0
rear = 2
Array = [10, 20, 30]
```

The valid queue is:

```
20, 30
```

The first valid element is:

```
Q[front + 1] = Q[1] = 20
```

51. Mistake 4: Using `front == -1` as the Only Empty Check

In this design, the empty condition is:

```
front == rear
```

Do not only check:

```
front == -1
```

Why?

Because after some deletions, the queue can become empty when both `front` and `rear` are equal at another value.

Example:

```
front = 2
rear = 2
```

This queue is empty even though `front` is not `-1`.

---

52. Mistake 5: Forgetting That Deleted Values May Still Exist Physically

After a two-pointer dequeue, the old value may still appear in the array.

But it is no longer part of the queue.

The queue only includes values from:

```
front + 1
```

to:

```
rear
```

This is why we say deletion is **logical**, not always physical.

---

53. Mistake 6: Confusing Index Pointers with Memory Address Pointers

In this phase, `front` and `rear` are array indexes.

Example:

```
front = 1
rear = 3
```

This means positions in the array.

It does not mean actual memory addresses.

---

54. Mistake 7: Forgetting the Full Condition

In this simple linear array queue, the queue is full when:

```
rear == size - 1
```

Example:

```
size = 5
rear = 4
```

The queue is full because index `4` is the last valid index.

Part F: Key Terms

55. Important Terms from Phase 2

Term	Simple Meaning
Array queue	A queue stored inside an array

Term	Simple Meaning
Single-pointer queue	A queue that only uses `rear`
Two-pointer queue	A queue that uses both `front` and `rear`
`rear`	Index of the last inserted element
`front`	Index before the first valid element in the two-pointer design
Shifting	Moving elements one position left or right
Logical deletion	Removing a value by moving `front`, even if the old value remains in memory
Empty condition	`front == rear`
Full condition	`rear == size - 1`
`O(1)`	Constant time
`O(n)`	Time depends on number of elements

Part G: Quick Review Questions

56. Question 1

In a single-pointer queue, which variable is used?

Answer:

```
rear
```

---

57. Question 2

Why is single-pointer deletion slow?

Answer:

```
Because after deleting index 0, all remaining elements must shift left.
```

---

58. Question 3

What is the time complexity of single-pointer insertion?

Answer:

```
O(1)
```

---

59. Question 4

What is the time complexity of single-pointer deletion?

Answer:

```
O(n)
```

---

60. Question 5

Which two variables are used in a two-pointer queue?

Answer:

```
front and rear
```

---

61. Question 6

In this design, what does `front` point to?

Answer:

```
front points before the first valid element.
```

---

62. Question 7

What is the empty condition in a two-pointer queue?

Answer:

```
front == rear
```

---

63. Question 8

What is the full condition in this simple linear array queue?

Answer:

```
rear == size - 1
```

---

64. Question 9

Why is two-pointer deletion fast?

Answer:

```
Because it moves front forward instead of shifting elements.
```

---

65. Question 10

If `front = 0`, `rear = 2`, and the array is `[10, 20, 30]`, what values are logically in the queue?

Answer:

```
20, 30
```

Why?

Because valid elements start from:

```
front + 1 = 1
```

and go to:

```
rear = 2
```

---

66. Final Phase 2 Summary

Phase 2 teaches how queues can be stored inside arrays.

There are two designs:

1. **Single-pointer queue**
2. **Two-pointer queue**

The single-pointer queue uses only `rear`.

Insertion is fast:

```
O(1)
```

But deletion is slow:

```
O(n)
```

because elements must shift left.

The two-pointer queue uses both `front` and `rear`.

Insertion is fast:

```
O(1)
```

Deletion is also fast:

```
O(1)
```

because we only move `front` forward.

The most important idea is:

> The two-pointer queue avoids shifting by using `front` to track logical deletion.

Remember these conditions:

```
front == rear        // queue is empty
rear == size - 1     // queue is full in a linear array queue
```

And remember:

```
front + 1 is the first valid element
rear is the last valid element
```

---

67. Final Takeaway

A single-pointer queue works, but it is inefficient for deletion.

A two-pointer queue is better because it removes values by moving `front`, not by shifting the whole array.

So the key Phase 2 lesson is:

```
Single pointer: simple but slow deletion.
Two pointers: better and faster deletion.
```

This prepares you for Phase 3, where the two-pointer design becomes a complete C program with `create`, `enqueue`, `dequeue`, and `display` functions.